

Homework 06 — STAT 432 (Fall 2026)

Name: Owen Park

NetID: owenp3

Questions 1 and 2 are done in Python (numpy 1.26, pandas, sklearn 1.6). The 70/30 split is the one shipped with the assignment in `breast-cancer-split.csv` (398 training, 171 test rows, generated in R with seed 432), so that the same split is used for every model in both questions and the numbers are comparable with the posted solutions. Predictors are standardized with the training means and standard deviations (`ddof=1`). KNN uses Euclidean distance, equal neighbor weights, and brute-force search; logistic regression has an intercept and no penalty. Question 3 is the peer evaluation; part 2 of it is written by me without AI help.

```
In [1]: import warnings
warnings.filterwarnings("ignore")
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.linear_model import LogisticRegression
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import roc_curve, roc_auc_score
pd.set_option("display.width", 120)

data = pd.read_csv("data/breast-cancer.csv")
split = pd.read_csv("data/breast-cancer-split.csv")
train = data.loc[split["split"].eq("train")]
test = data.loc[split["split"].eq("test")]
predictors = ["mean_radius", "mean_texture", "mean_smoothness"]

x_bar = train[predictors].mean()
s = train[predictors].std(ddof=1)
X_train = ((train[predictors] - x_bar) / s).to_numpy()
X_test = ((test[predictors] - x_bar) / s).to_numpy()
y_train = train["y"].to_numpy()
y_test = test["y"].to_numpy()
print(f"train n = {len(y_train)} (malignant {y_train.sum()}), test n = {len(y_test)} (malignant {y_test.sum()})")
print("training means:", x_bar.round(4).to_dict())
print("training sds: ", s.round(4).to_dict())
print("test columns after standardizing, mean/sd:", X_test.mean(0).round(3), X_test.std(0, ddof=1).round(3))

def knn(k):
    return KNeighborsClassifier(n_neighbors=int(k), weights="uniform", metric="euclidean", algorithm="brute")

train n = 398 (malignant 147), test n = 171 (malignant 65)
training means: {'mean_radius': 14.1952, 'mean_texture': 19.2309, 'mean_smoothness': 0.0962}
training sds: {'mean_radius': 3.5572, 'mean_texture': 4.285, 'mean_smoothness': 0.0139}
test columns after standardizing, mean/sd: [-0.064 0.046 0.048] [0.97 1.015 1.036]
```

Question 1: Compare KNN and logistic regression

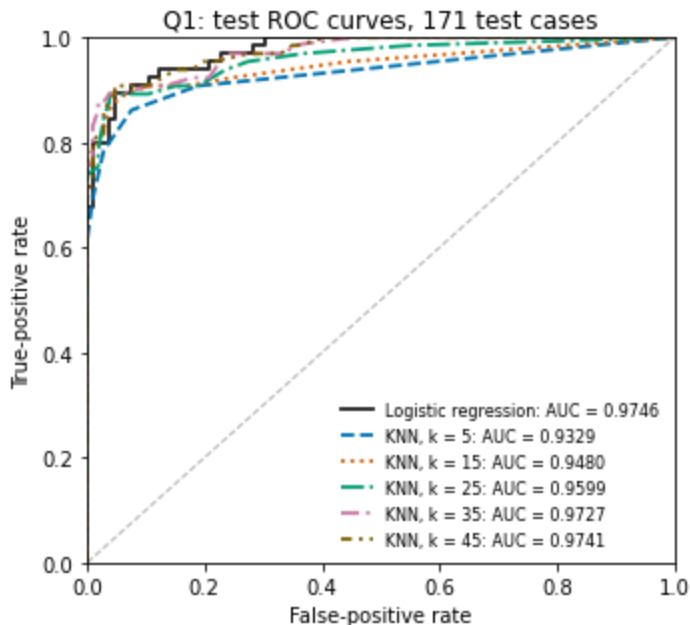
Six models on the same split: logistic regression (intercept, no penalty) and KNN with $k = 5, 15, 25, 35, 45$. The KNN probability of `y = 1` is the share of malignant cases among the k nearest training points, which is what `predict_proba` returns with equal weights. All ROC curves and AUCs are computed from the test probabilities.

```
In [2]: logit = LogisticRegression(C=np.inf, solver="lbfgs", fit_intercept=True, tol=1e-10, max_iter=1000).fit(X_train, y_train)
print("logistic coefficients (standardized scale):", dict(zip(["intercept"] + predictors, np.r_[logit.intercept_, logit.coef_].T)))

ks1 = [5, 15, 25, 35, 45]
names = ["Logistic regression"] + [f"KNN, k = {k}" for k in ks1]
P1 = np.column_stack([logit.predict_proba(X_test)[:, 1]] + [knn(k).fit(X_train, y_train).predict_proba(X_test)[:, 1] for k in ks1])
auc1 = np.array([roc_auc_score(y_test, P1[:, m]) for m in range(len(names))])
print(pd.DataFrame({"Model": names, "Test AUC": auc1.round(4)}).to_string(index=False))
```

```
logistic coefficients (standardized scale): {'intercept': -1.17, 'mean_radius': 5.327, 'mean_texture': 1.786, 'mean_smoothness': 2.148}
Model Test AUC
Logistic regression 0.9746
KNN, k = 5 0.9329
KNN, k = 15 0.9480
KNN, k = 25 0.9599
KNN, k = 35 0.9727
KNN, k = 45 0.9741
```

```
In [3]: colors = ["#222222", "#0072B2", "#D55E00", "#009E73", "#CC79A7", "#806000"]
styles = ["-", "--", ":", "-.", (0, (5, 2, 1, 2)), (0, (3, 2, 1, 2, 1, 2))]
fig, ax = plt.subplots(figsize=(5.0, 4.6))
ax.plot([0, 1], [0, 1], color="0.7", linestyle="--", linewidth=1)
for m, name in enumerate(names):
    fpr, tpr, _ = roc_curve(y_test, P1[:, m])
    ax.plot(fpr, tpr, color=colors[m], linestyle=styles[m], linewidth=1.8, label=f"{name}: AUC = {auc1[m]:.4f}")
ax.set(xlim=(0, 1), ylim=(0, 1), xlabel="False-positive rate", ylabel="True-positive rate", title="Q1: test ROC cur")
ax.legend(loc="lower right", frameon=False, fontsize=8); plt.tight_layout(); plt.show()
```



On this split logistic regression has the largest test AUC, 0.9746, and that is the model I prefer. KNN with $k = 45$ is essentially tied at 0.9741 and $k = 35$ is close at 0.9727, while the small neighborhoods rank the test cases noticeably worse: 0.9329 for $k = 5$, 0.9480 for $k = 15$, 0.9599 for $k = 25$. The ROC curves tell the same story: the $k = 5$ curve sits below the others over most of the false-positive range, and the three best curves are hard to separate by eye. I pick logistic regression over $k = 45$ not because 0.0005 of AUC means anything, but because it ties on ranking while being a fixed three-coefficient model: the standardized coefficients (radius 5.3, smoothness 2.1, texture 1.8) say directly what drives the risk, it needs no stored training set, and it does not depend on a tuning constant. If I had to pick a k , the trend toward larger neighborhoods suggests the decision boundary here is smooth enough that a near-linear fit does well. Two caveats: this is one 171-case test set, so differences in the third decimal are noise, and AUC measures ranking, not accuracy at the cutoff a clinic would actually use. Choosing among models properly would mean cross-validating k on the training data and keeping the test set for the final check only.

Question 2: Training and test ROC curves

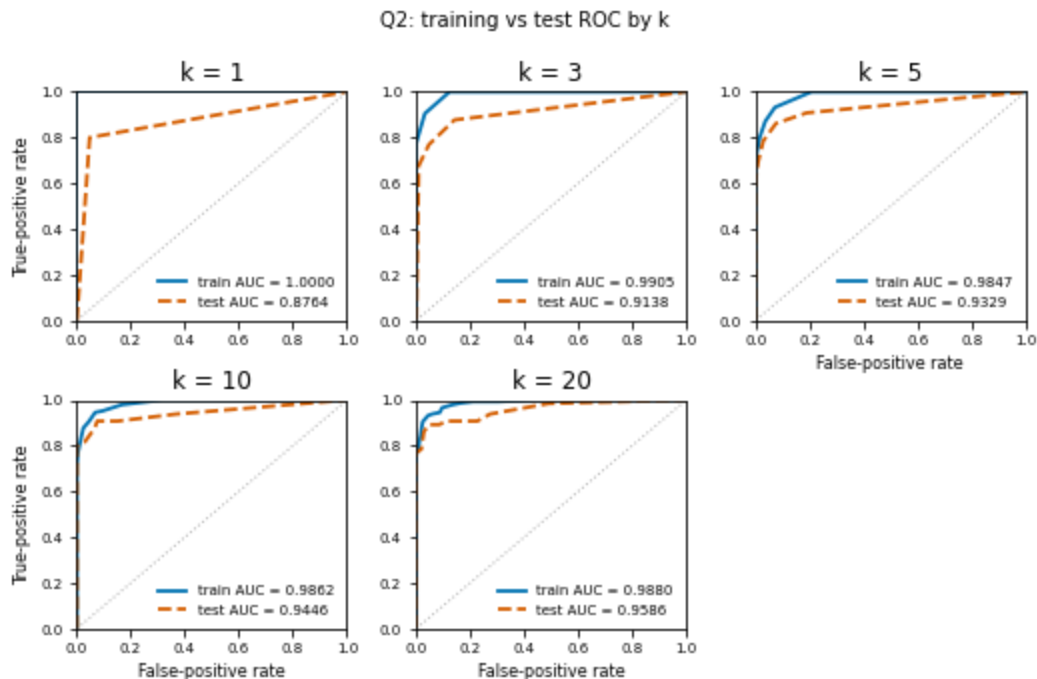
Same split and standardization. KNN with $k = 1, 3, 5, 10, 20$, probabilities for both the training rows (each row is allowed to be its own neighbor, which is what happens when `X_train` is passed to `predict_proba`) and the test rows.

```
In [4]: ks2 = [1, 3, 5, 10, 20]
Ptr = np.empty((len(y_train), len(ks2))); Pte = np.empty((len(y_test), len(ks2)))
for m, k in enumerate(ks2):
    fit = knn(k).fit(X_train, y_train)
    Ptr[:, m] = fit.predict_proba(X_train)[: , 1]
    Pte[:, m] = fit.predict_proba(X_test)[: , 1]
auc_tr = np.array([roc_auc_score(y_train, Ptr[:, m]) for m in range(len(ks2))])
auc_te = np.array([roc_auc_score(y_test, Pte[:, m]) for m in range(len(ks2))])
tab2 = pd.DataFrame({"k": ks2, "Training AUC": auc_tr.round(4), "Test AUC": auc_te.round(4), "gap": (auc_tr - auc_te).round(4)})
print(tab2.to_string(index=False))
# 1NN check: probabilities are 0/1, so the test AUC is (sensitivity + specificity)/2
p1 = Pte[:, 0]; sens = p1[y_test == 1].mean(); spec = 1 - p1[y_test == 0].mean()
print(f"1NN test: distinct probabilities {np.unique(p1)}, sensitivity {sens:.4f}, specificity {spec:.4f}, (sens+spec)/2 = {sens+spec}/2 = 0.8764")
print(f"1NN training: fitted probability equals the label for {int((Ptr[:, 0] == y_train).sum())} of {len(y_train)} rows")
```

k	Training AUC	Test AUC	gap
1	1.0000	0.8764	0.1236
3	0.9905	0.9138	0.0768
5	0.9847	0.9329	0.0518
10	0.9862	0.9446	0.0415
20	0.9880	0.9586	0.0294

1NN test: distinct probabilities [0. 1.], sensitivity 0.8000, specificity 0.9528, (sens+spec)/2 = 0.8764
1NN training: fitted probability equals the label for 398 of 398 rows

```
In [5]: fig, axes = plt.subplots(2, 3, figsize=(7.4, 4.9))
for m, k in enumerate(ks2):
    ax = axes.flat[m]
    fpr, tpr, _ = roc_curve(y_train, Ptr[:, m]); ax.plot(fpr, tpr, color="#0072B2", linewidth=1.8, label=f"train AUC = {auc_tr[m]:.4f}")
    fpr, tpr, _ = roc_curve(y_test, Pte[:, m]); ax.plot(fpr, tpr, color="#D55E00", linestyle="--", linewidth=1.8, label=f"test AUC = {auc_te[m]:.4f}")
    ax.plot([0, 1], [0, 1], color="0.7", linestyle=":", linewidth=1)
    ax.set(title=f"k = {k}", xlim=(0, 1), ylim=(0, 1)); ax.tick_params(labelsize=7)
    if m >= 2: ax.set_xlabel("False-positive rate", fontsize=8)
    if m % 3 == 0: ax.set_ylabel("True-positive rate", fontsize=8)
    ax.legend(loc="lower right", frameon=False, fontsize=7)
axes.flat[-1].set_visible(False); fig.suptitle("Q2: training vs test ROC by k", fontsize=10); plt.tight_layout(); p
```



The gap between training and test performance is largest at $k = 1$ and shrinks as k grows: training AUC 1.0000 against test AUC 0.8764 at $k = 1$ (gap 0.124), then gaps of 0.077, 0.052, 0.042 and 0.029 at $k = 3, 5, 10, 20$, with the test AUC climbing steadily to 0.9586. At $k = 1$ the training result is not evidence of anything: each training point is its own nearest neighbor at distance zero, so the fitted probability is simply its own label, and I checked that this holds for all 398 rows. A ranking that reproduces the labels exactly must have AUC 1. The test rows have no such shortcut, and because 1NN outputs only 0 or 1, its test ROC curve is one corner point and its AUC is just $(\text{sensitivity} + \text{specificity})/2 = (0.800 + 0.953)/2 = 0.8764$. Increasing k dilutes the weight of any single label, including a row's own label in the training predictions, so the training AUC drops to about 0.985 to 0.988 and stops being perfect, while the test AUC improves because averaging over more neighbors cancels the label noise of individual cases. The training AUC is not monotone (it dips at $k = 5$ and edges back up at 10 and 20),

which is a reminder that smoothing can improve ranking on both sets at once. Training performance overstates what the model will do on new data whenever the fit can memorize the labels it is scored on; the smaller k , the more memorizing a KNN fit does, so the more its training ROC curve flatters it.

Question 3: Peer evaluation of Homework 5

The two reports assigned to me in my repository's Issue #1 are by PrerithArunkumar and stanlyyp (both `homework/week-05/homework-05.pdf`).

3.1 The item I selected

Report: stanlyyp. **Question:** Homework 5, Question 2(a), the table of test MSE values for 5NN and the lasso in the two covariate settings. The table reports a 5NN test MSE of 2.126 and a lasso test MSE of 2.065 in Setting 1 (independent covariates), and 0.754 and 1.934 in Setting 2 (correlation 0.8). Those numbers cannot come from the model in the assignment. With noise standard deviation 0.1 the response has variance of about 0.9 in Setting 1 and about 1.3 in Setting 2 (the correlated covariates add covariance between the three signal terms), so predicting every test case with the training mean already gives test MSEs near 0.9 and 1.3. The report's lasso is worse than that baseline in both settings (2.065 and 1.934), and its Setting 1 5NN (2.126) is too. A lasso with a fitted intercept, selected by cross-validation, cannot lose to the training mean by a factor of two on 1,000 test points. Something upstream of the table is off (the noise level, the centering of the response, or the scaling of the test data), and the written interpretation in 2(b) and 2(c) is built on these numbers. The code is not shown in the report, so the reader cannot tell which.

3.2 My assessment (written without AI assistance)

The table I am questioning, quoted from Question 2(a) of the report:

Setting 1 (Independent): KNN Test MSE 2.126, Lasso Test MSE 2.065, Selected λ 0.1334, Nonzero Slopes 2
Setting 2 (Correlated, $\rho=0.8$): KNN Test MSE 0.754, Lasso Test MSE 1.934, Selected λ 0.0422, Nonzero Slopes 8

The answer is wrong because the numbers cannot be possible for the model in the assignment. The report's lasso and Setting 1 5NN are worse than just using the simple mean of the training y as the prediction, which makes no sense: using a model is worse than just picking the mean. The problem might be not adding the training mean of y back to the lasso prediction, or standardizing the test covariates with their own means and scales instead of the training ones, but I can't tell because the code is not shown there.

3.3 A correction

Below is a corrected Question 2(a) in Python, following the assignment: 400 training and 1,000 test observations per setting, 5NN on the raw covariate scale, and the lasso on

centered and standardized covariates with a separately fitted intercept, penalty chosen by ten-fold cross-validation over 41 log-spaced values from $1e-4$ to 1, with means and scales estimated on each fold's training part only. `sklearn.linear_model.Lasso` minimizes exactly the Week 4 objective, $(1/2n)\|y - Xb\|^2 + \lambda \|b\|_1$. I first print the variance of Y , which is the test MSE of the "predict the mean" baseline that any sensible fit must beat.

```
In [6]: from sklearn.linear_model import Lasso
from sklearn.neighbors import KNeighborsRegressor

def f5(X): return 0.5*X[:, 0] + np.sin(X[:, 1]) - 0.3*X[:, 2]**2
rng = np.random.default_rng(43262)
lams = np.logspace(0, -4, 41)

def gen(n, rho):
    Z = rng.standard_normal((n, 30))
    X = np.sqrt(1 - rho)*Z + np.sqrt(rho)*rng.standard_normal((n, 1)) if rho > 0 else Z
    return X, f5(X) + rng.normal(0, 0.1, n)

def lasso_path_fit(X, y, lam_list):
    m = X.mean(0); s = X.std(0, ddof=0); ym = y.mean() # divisor = sample size of the data they are estimating
    Xs = (X - m)/s
    fits = [Lasso(alpha=l, fit_intercept=False, max_iter=50000, tol=1e-8).fit(Xs, y - ym) for l in lam_list]
    return m, s, ym, fits

def lasso_cv_fit(X, y):
    folds = rng.permutation(np.arange(len(y)) % 10)
    cv = np.zeros(len(lams))
    for v in range(10):
        tr, va = folds != v, folds == v
        m, s, ym, fits = lasso_path_fit(X[tr], y[tr], lams)
        Xv = (X[va] - m)/s
        cv += np.array([np.mean((ym + fit.predict(Xv) - y[va])**2) for fit in fits])/10
    j = int(np.argmin(cv))
    m, s, ym, fits = lasso_path_fit(X, y, [lams[j]])
    return lams[j], m, s, ym, fits[0]

rows = []
for setting, rho in [(1, 0.0), (2, 0.8)]:
    Xtr, ytr = gen(400, rho); Xte, yte = gen(1000, rho)
    var_y = np.mean((yte - ytr.mean())**2)
    knn5 = np.mean((KNeighborsRegressor(5, weights="uniform", algorithm="brute").fit(Xtr, ytr).predict(Xte) - yte)**2)
    lam, m, s, ym, fit = lasso_cv_fit(Xtr, ytr)
    lasso_mse = np.mean((ym + fit.predict((Xte - m)/s) - yte)**2)
    rows.append({"setting": setting, "rho": rho, "test MSE of the mean (~Var Y)": var_y, "5NN test MSE": knn5,
                "lasso test MSE": lasso_mse, "selected lambda": lam, "nonzero slopes": int((fit.coef_ != 0).sum())})
print(pd.DataFrame(rows).round(4).to_string(index=False))
```

setting	rho	test MSE of the mean (~Var Y)	5NN test MSE	lasso test MSE	selected lambda	nonzero slopes
1	0.0	0.9327	0.5964	0.2611	0.0501	3
2	0.8	1.3626	0.1715	0.2107	0.0126	8

With a correct fit both methods beat the mean baseline by a wide margin: 5NN is the worse method with independent covariates (its distance is dominated by the 27 noise coordinates) and the better one under correlation 0.8 (the shared factor makes neighbors informative), while the lasso sits near 0.25 to 0.3 in both settings because it cannot represent $\sin(x_2)$ and x_3^2 and has a floor set by that misspecification. These are the same qualitative conclusions the report states in 2(b) and 2(c), so the interpretation survives; the table does not. A test MSE above $\text{Var}(Y)$ is the quickest check to apply to any table like this, and the fix is to print that baseline next to the model errors and to show the simulation code so the reader can find which step is off (noise sd 0.1, not 1; center y with the training mean and add it back; standardize the test covariates with the training means and scales).

GitHub Issue: <https://github.com/stanlyyp/stat432-fall2026/issues/2>